

Available online at [www.sciencedirect.com](http://www.sciencedirect.com)

ScienceDirect

journal homepage: [www.elsevier.com/locate/cose](http://www.elsevier.com/locate/cose)Computers  
&  
Security

## TC 11 Briefing Papers



## Data integrity auditing for secure cloud storage using user behavior prediction

Junfeng Tian<sup>a,b</sup>, Haoning Wang<sup>a,b,\*</sup>, Meng Wang<sup>a,b</sup><sup>a</sup> School of Cyberspace Security and Computer Institute, Hebei University, Baoding 071000, China<sup>b</sup> Hebei Key Laboratory of High Confidence Information Systems (Hebei University), Baoding 071000, China

## ARTICLE INFO

## Article history:

Received 7 October 2020

Revised 4 January 2021

Accepted 21 February 2021

Available online 26 February 2021

## Keywords:

Data integrity

Cloud storage

User behavior prediction

Valid auditing

Data sharing

## ABSTRACT

Data integrity is a core security issue in reliable cloud storage that has received widespread attention. Data auditing protocols enable verifiers to efficiently check the integrity of out-sourced data without downloading the data. A key research challenge associated with the design of existing data auditing protocols is the efficiency of the auditing process. Since existing protocols tend to auditing all cloud data, in fact, some data may have just been used or will be used soon, then auditing all these data is invalid, which is a waste of resources. In this paper, we attempt to address the waste of resources due to invalid auditing in cloud data integrity checking by introducing user behavior prediction algorithms, the first in such an approach, to the best of our knowledge. More specifically, we introduce the concept of valid auditing in data integrity verification based on the concept of valid auditing. We formalize a system model and a security model for this new concept. Then, using user behavior prediction algorithms, we give methods to improve auditing efficiency and reduce invalid auditing. Finally, we develop a prototype implementation of the protocol to validate the practicality of the scheme.

© 2021 Elsevier Ltd. All rights reserved.

## 1. Introduction

With the continuous improvement and development of cloud storage technology, it has been accepted by more and more institutions, companies and individuals, and has become a IaaS service in cloud computing Hogan et al. (2011). Users upload data generated by various heterogeneous devices to the cloud for analysis, processing and long-term storage Dewan and Hansdah (2011). However, data stored in the cloud may be lost due to software errors or maliciously damaged. In order to

confirm whether the data is stored in the cloud intact, the researchers proposed the auditing scheme for cloud storage.

In data auditing schemes, data owners first generates a corresponding signature for each data block. These signatures are used to verify whether the data is stored the cloud intact. Then the data owner uploads the data and the data signature to the cloud. In cloud storage applications, cloud data is often used to share with other users. There are already many mature applications, such as Google Drive, iCloud, Aliyun, etc. Data sharing has become a basic feature of cloud storage, allowing data owners to share data with other users. However, as the scale of shared data continues to increase and the number of users

\* Corresponding author.

E-mail address: [479601516@qq.com](mailto:479601516@qq.com) (H. Wang).<https://doi.org/10.1016/j.cose.2021.102245>

0167-4048/© 2021 Elsevier Ltd. All rights reserved.

sharing system services continues to increase, the throughput of the cloud has gradually become one of the main issues that restrict the efficiency of the data sharing system. A typical application of data sharing system is remote office system (ROS) [Igeltjorn and Habib \(2020\)](#). In ROS, employees of branches located in various places can upload data to the cloud to share data and collaborate with colleagues in different places. If all users download the required data directly from the cloud, the throughput of the cloud may cause a large number of users to download slowly. In addition, since software and hardware failures or human errors may occur in the cloud, the integrity of ROS data needs to be guaranteed. Therefore, how to reduce user waiting time and ensure the integrity of cloud data is very important.

An obvious way to think about this is to increase the throughput of the cloud and reduce the wait. However, in the real world, access to peak periods tend to be more concentrated, just increase the throughput will cause a waste of resources and higher costs [Ma et al. \(2020\)](#). Another method is to download data frequently used by users to the local in advance, which can reduce the amount of downloads by users in the cloud during peak periods and reduce waiting time. However, this method also has limitations, because the needs of users are often dynamic, and if the amount of pre-downloaded data is too large and the user's storage space requirements are also too high, then the meaning of cloud storage will be lost. The idea of this method is desirable, but it needs to solve the two problems of user dynamic demand and excessive pre-downloading. In addition, existing schemes often audit all of the user's data, but the integrity of much of this data has already been or will be assured, so an audit of these data is not necessary, such as files that the user has just accessed and files that the user is about to access. These documents may have passed the audit of these documents is unnecessary, resulting in a waste of resources. Therefore, while ensuring the security of cloud data, it is very important to provide users with efficient and convenient data sharing solutions. Unfortunately, previous studies have often ignored this issue and have not provided an effective scheme.

### 1.1. Contribution

The contributions of this paper are summarized as follows.

- (1) In this paper, a data auditing scheme based on user behavior prediction is proposed. The proposed scheme is designed with reference to the cloud-fog-terminal architecture proposed by Cisco [Bonomi et al. \(2012\)](#), and security analysis and experimental results show that the proposed scheme is secure and efficient.
- (2) We introduce a user behavior prediction algorithm to load the data that users might use according to their historical behavior characteristics to the fog node serving them in advance, which solves the problem of throughput shortage in the cloud during peak access periods and improves user experience.
- (3) We introduce the concept of valid auditing. Choosing data that is in the valid auditing period for auditing reduces the auditing overhead, especially in a shared system with a low proportion of active files.

### 1.2. Related works

In recent years, the security of cloud data has received increasing attention [Ren et al. \(2012\)](#). In order to verify the security of cloud data, cloud data integrity schemes have been proposed. In 2007, one of the earliest audit schemes, "proof of retrievability (PoRs)", was proposed [Juels and Kaliski \(2007\)](#). In the same year, Ateniese et al. proposed one of the earliest shared auditing schemes, provable data possession (PDP) [Ateniese \(2007\)](#). The scheme applied RSA-based homomorphic tags to ensure data integrity by simply remotely verifying partially randomly selected blocks of data. This is a pioneering effort that many subsequent cloud audits have continued to build upon [Wang et al. \(2013\)](#). As mentioned above, public auditing provides more credible results than private auditing and reduces unnecessary computational burdens on users by introducing third-party auditors (TPAs). As a result, public auditing is considered to be a more reliable and feasible auditing scheme [Tian \(2017\)](#). In addition to this, there are some other features that are considered important such as privacy protection, data sharing, batch auditing, and dynamic updates.

To solve the problem of data leakage, Wang et al. first proposed a privacy-preserving audit scheme based on homomorphic verifier and random masking techniques [Wang et al. \(2011b\)](#). Worku et al. proposed a data auditing scheme [Worku et al. \(2014\)](#) that also used random masks, which reduced the computational overhead of the users and made the scheme more feasible compared to the scheme [Wang et al. \(2011b\)](#). Third-party mediums (TPMs) are used to help users generate signatures without leaking user data.

To address the dynamic update problem, Erway extended the PDP scheme to propose an auditing scheme that supports dynamic update of all data using the skip table [Erway et al. \(2009\)](#). Later, Wang et al. proposed a dynamic data audit scheme using the Merkel hash tree [Wang et al. \(2011a\)](#), which adds support for batch auditing in addition to the dynamic update of data. However, the overhead of update operations in the above schemes are large and not well adapted to nowadays mobile demands [Yu and Wang \(2017\)](#). Shen et al. proposed an auditing scheme [Shen et al. \(2017\)](#) with a new dynamic structure that reduces the overhead of update operations by using double linked information table (DLIT) and location array (LA).

With the development of IoT, data sharing systems contain more and more heterogeneous devices. In order to ensure that these heterogeneous devices work together in the system, Cisco proposed the idea of cloud-fog computing [Bonomi et al. \(2012\)](#). Tian et al. proposed a cloud-fog computing based data auditing scheme [TIAN and NAN \(2019\)](#), which reduces the computational overhead for users and enables even low computational power devices to generate signatures for data and upload them to the cloud. Huang et al. proposed a joint cloud that enables dynamic data updates and A decentralized auditing scheme is completed through blockchain [Huang \(2020\)](#).

Based on the above, this paper explores how to reduce user access latency during peak access periods and how to improve the effectiveness of auditing. Compared with existing schemes [Huang \(2020\)](#); [Li et al. \(2019\)](#); [Shen et al. \(2019\)](#), the proposed

**Table 1 – NOTATIONS.**

| Notation                             | Meaning                                                 |
|--------------------------------------|---------------------------------------------------------|
| $p$                                  | One large prime                                         |
| $G, G_T$                             | The multiplicative cyclic groups with order $p$         |
| $e$                                  | A bilinear pairing map $e : G \times G \rightarrow G_T$ |
| $Z_p^*$                              | A prime field with nonzero elements                     |
| $H$                                  | A hash function: $H : \{0, 1\}^* \rightarrow G$         |
| $g$                                  | A generator of $G$                                      |
| $\mu', \mu$                          | The elements of $G$                                     |
| $F = \{m_1, m_2, \dots, m_n\}$       | Original data file                                      |
| $ID$                                 | The identity information                                |
| $\Phi = \{\sigma_i\}_{i \in [1, n]}$ | The signature of file $F$                               |
| $\Delta T$                           | One recommended period                                  |
| $Count_{\Delta T}$                   | One pre-set threshold within $\Delta T$                 |

scheme uses only 15–20% of the number of challenge blocks to achieve the same auditing effect.

### 1.3. Organization

The structure of this paper is as follows. Chapter 2 introduces the meaning and preparatory knowledge of the symbols used in this paper. Chapter 3 introduces the system and security model. Chapter 4 introduces the document status table and valid auditing first proposed in this paper. Chapter 5 is a detailed description of the proposed scheme. Chapter 6 is the security analysis. Chapter 7 is a presentation of the experimental results. Chapter 8 provides a summary of this paper.

## 2. Preliminaries and problem statement

### 2.1. Notations

Some of the symbols used in the proposed scheme are shown in Table 1.

### 2.2. Preliminaries

This section introduces some cryptography, including bilinear pairing and the Computational Diffie-Hellman(CDH) problem and Discrete Logarithm(DL) problem. Bilinear pairing is an important technology in modern cryptography. It is often used in encryption/decryption algorithms since 2001, when an identity-based encryption was proposed by Boneh et al. (2001). Certainly, bilinear pairing has become one of major techniques for signature verification in cloud auditing schemes. The CDH problem and the DL problem are important guarantees for signature resistance to cracking. It is generally believed that the difficulty of the CDH problem depends on the DL problem. The DL problem cannot be solved in polynomial time, so the signature is security and reliable.

#### 2.2.1. Bilinear map

Let  $G$  and  $G_T$  be two multiplicative cyclic groups of same large prime order  $p$ , and  $g$  be a generator of  $G$ . Bilinear map is a map  $e : G \times G \rightarrow G_T$  with the following properties: a) Bilinearity: for all  $g_1, g_2 \in G$  and  $a, b \in Z_p^*$ ,

$e(g_1^a, g_2^b) = e(g_1, g_2)^{ab}$  b) Computability: there exists an efficiently computable algorithm for computing map  $e$ . c) Non-degeneracy:  $e(g, g) \neq 1$ .

#### 2.2.2. Computational diffie-Hellman(CDH) problem

For unknown  $x, y \in Z_p^*$ , given that  $g$  and  $g^x$  are the inputs,  $g^{xy} \in G$  are the outputs. The CDH assumption in  $G$  holds if solve the CDH problem in  $G$  is computationally infeasible.

#### 2.2.3. Discrete logarithm(DL) problem

For unknown  $x \in Z_p^*$ , given  $g, g^x$  as input, outputs  $x$ . The DL assumption in  $G$  holds if it is computationally infeasible to solve the DL problem in  $G$ .

### 2.3. System model

The system model of the proposed scheme is shown in Fig. 1.

User: Users consist of Data Owners (DO) and Shared Users. Data owners can upload the data collected and processed by the devices under their control to the cloud for use by the shared users in the system. In a data sharing system, the identity of the data owner and the Shared Users are not stable. In other words, a shared user becomes a data owner when it uploads data, and a data owner becomes a shared user when it uses data uploaded by other users.

CSP: A cloud service provider provides massive storage space to users. In a shared system, the data between DOs is not independent of each other because the cloud service provider stores data on a system basis. At the same time, the CSP runs a prediction algorithm and then sends the recommended files to the fog center that serves the user.

TPA: TPA is a trusted third-party auditor that helps users audit the data stored in the CSP. Any user in the shared system can authorize the TPA to challenge the CSP and audit the files in the shared system.

FCC: A fog compute center contains multiple fog nodes, which are set up by the operator where the user is located and provide services for users in the area. The fog compute center stores the user's recommended files and updates the recommended files based on the user's usage.

PKG: The PKG generates the private key for the user based on the identity ID and is a semi-trusted entity, which means that the user has to verify that the private key is correct. The user sends the ID information to PKG, and PKG generates the private key for the user based on the ID information. After receiving the private key, the user checks the correctness of the private key and then uses the private key and system parameters to generate file signatures, and then sends the file signatures and files to the cloud for long-term storage. According to the user's access habits, the cloud recommended files for the user and sent to the user's fog center in the area. The user can access the recommended files directly from the fog center, and the TPA audits the files that are not within the auditing hold period based on the file usage priority. Valid auditing details can be found in Chapter 3, and scheme details can be found in Chapter 4.

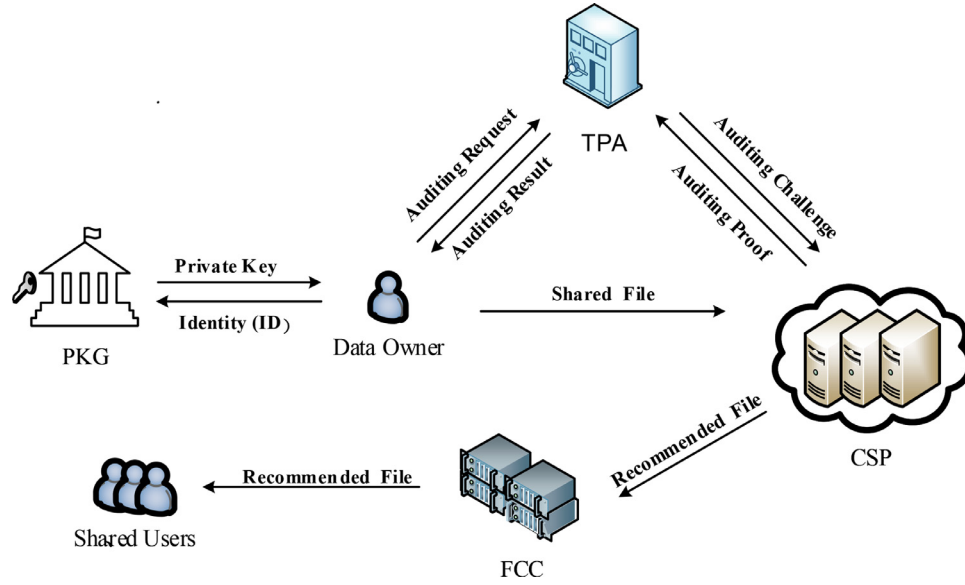


Fig. 1 – System Model.

## 2.4. Design goals

In order to ensure the efficiency and security of user behavior prediction-based data integrity auditing for secure cloud storage, the proposed scheme should achieve the following goals

### 2.4.1. The correctness

- Private key correctness: The user should first verify the private key sent by the PKG to ensure the correctness of the private key.
- Auditing correctness: If the cloud stores the user's data intact, the auditing proof should certainly pass the validation.
- The correctness of valid auditing: To ensure the correctness of an effective auditing, the recommended files should be as accurate as possible.

### 2.4.2. The soundness

- Auditing soundness: If the cloud does not store the user's data intact, then the signature it generates cannot pass the TPAs validation.
- Soundness of recommended file: The recommended file should be accurate enough to ensure the reliability of the system.

## 2.5. Algorithms of the proposed scheme

**Definition 1.** The proposed scheme consists of three steps: *FileUpload*, *FileRec* and *FileVerify*. where *FileUpload* consists of three algorithms: *Setup*, *KeyGen* and *SigGen*. *FileRec* consists of two algorithms: *FileAct* and *Usersim*. *FileVerify* consists of three algorithms: *ChalGen*, *ProofGen* and *ProofVerify*. the algorithms are described as follows.

- Setup*( $1^k$ ) It is a setup algorithm run by PKG. Input security parameter  $k$ , output master key  $msk$  and system parameters  $pp$ .

- KeyGen*( $pp, ID$ ) It is a secret key generation algorithm run by PKG. Input system parameters  $pp$ , master key  $msk$  and user identity  $ID$  and output private key  $sk$ .
- SigGen*( $F, msk, ssk, name$ ) It is a signature generation algorithm run by the user. Input file  $F$ , user private key  $sk$ , signing private key  $ssk$  and file identity  $name$ . Output the signature  $\Phi$  of the file.
- FileAct*( $F$ ) It is a file activation algorithm run by the cloud. Input file  $F$ , output the file status table  $FST$ .
- UserSim*( $F, UserRcd$ ) It is a user similarity computation algorithm run by the cloud. Input file  $F$  and user behavior record table and output the prediction result.
- Chal*( $FST$ ) It is a challenge block generation algorithm run by the TPA with input  $FST$  and output challenge block  $chal$ .
- ProofGen*( $F, \Phi, chal$ ) It is a proof generation algorithm run by a CSP. It inputs file  $F$ , signature set  $\Phi$ , challenge block  $chal$ , and outputs auditing proof  $P$ .
- ProofVerify*( $chal, pp, P$ ) It is an auditing proof verification algorithm run by TPA. Input challenge block  $chal$ , system parameter  $pp$ , the auditing proof  $P$ , and output auditing result.

## 2.6. Security model

In order to formalize the safety model, the game of Adversary  $A$  and Challenger  $C$  was designed to show to show the adversary  $A$  is against the security of the proposed scheme. The game is described as follows.

- Setup phase*: Challenger  $C$  runs the *Setup* algorithm to get the private key  $sk$  and the public parameters  $PP$ , and then sends the public parameters  $PP$  to the adversary  $A$ . The challenger  $C$  runs the *Setup* algorithm to get the private key  $sk$  and the public parameters  $PP$ .
- Query phase*: In this phase, Adversary  $A$  will make the following two queries to Challenger  $C$ .



- **Extract Queries:** The adversary  $A$  sends the identity information  $ID$  to the challenger  $C$ . The challenger executes the *Extract* algorithm to obtain the private key  $sk$ , and then sends it to adversary  $A$ .
  - **SigGen Queries:** Challenger  $A$  asks for the corresponding signature of file  $F$ . Challenger  $C$  executes the *Extract* algorithm to obtain the private key. Challenger  $C$  then runs the *SigGen* algorithm to calculate the corresponding signature of file  $F$  and sends it to adversary  $A$ .
- (3) **Challenge phase:** In this phase, the adversary will act as a prover and the challenger will act as a verifier. The challenger  $C$  sends the challenge block  $chal = \{i, v_i\}_{i \in I}$  to adversary  $A$ , where  $I \in \{\omega_1, \omega_2, \dots, \omega_c\}_{\omega_j \in [1, n], j \in [1, c]}$ . The challenger asks the adversary to provide a proof of data possession  $P$  for the corresponding data block.
- **Extract queries:** The adversary can perform private key queries based on the identity information  $ID$ .
  - **SigGen Queries:** Challenger  $A$  asks for the corresponding signature of file  $F$ . The challenger  $C$  executes the *Extract* algorithm to obtain the private key. Challenger  $C$  then runs the *SigGen* algorithm to calculate the corresponding signature of file  $F$  and sends it to adversary  $A$ .
- (4) **ProofGenphase:** After receiving a challenge from the challenger  $C$ , the adversary  $A$  generates a proof  $P$  of the challenge block and sends it to the challenger  $C$ . If the proof  $P$  passes the verification with non-negligible probability, it is considered that the adversary  $A$  wins the game.

For the above security model, it needs to be shown that if the adversary  $A$  does not possess all the data blocks requested by the challenger, it cannot generate the correct proof  $P$ , which means that the proof forged by the adversary cannot pass verification. The goal of the adversary  $A$  is to correctly generate the corresponding signature of the challenge block  $chal$  without possessing the data block.

**Definition 2.** We say a remote data integrity auditing scheme is secure if the following condition holds: whenever the adversary  $A$  in the aforementioned game can generate a valid proof  $P$  to pass the verification of the challenger  $C$  with non-negligible probability, there exists a knowledge extractor that can capture the challenged data blocks except possibly with negligible probability.

**Definition 3.** A remote data integrity auditing scheme is  $(\rho, \delta)$  ( $0 < \rho, \delta < 1$ ) detectable if the cloud corrupted  $\rho$  fraction of the whole file, these corrupted data blocks can be detected with the probability at least  $\delta$ .

### 3. The proposed structure and concept

#### 3.1. The valid auditing

In practice, if a user has just accessed, audited, or is soon to access a file, there is no point in auditing the file, which is a waste of time and computing resources. In other words, auditing a file only when necessary. A valid auditing is an au-

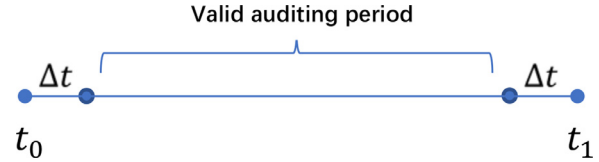


Fig. 2 – The valid auditing.

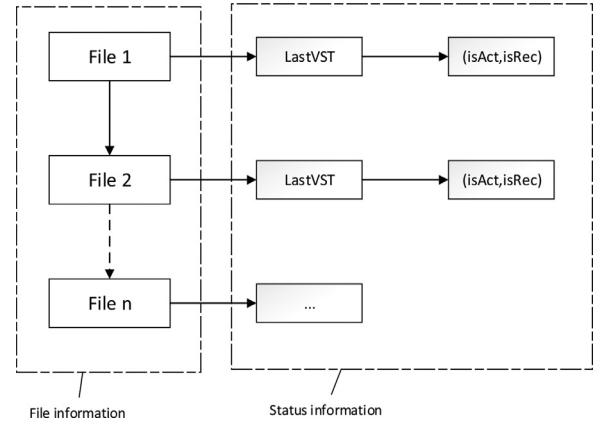


Fig. 3 – The file status list.

ditig that takes place during the valid auditing period. The valid auditing period is defined as shown in Fig. 3.  $P_{ea}$  denotes the valid auditing rate,  $N_e$  denotes the number of the amount of valid auditing, and  $N_a$  denotes the total number of audits.  $t_0$  denotes the time of the most recent auditing or access to the file.  $t_1$  denotes the time of the next access to the file.  $\Delta t$  is the auditing retention period, during which the file has the highest probability of remaining intact and can be considered approximately intact. In practice,  $\Delta t$  should be a dynamically adjusted value. When a user accesses a file frequently, the  $\Delta t$  of that file increases and decreases otherwise. The effective audit rate  $P_{ea} = N_e/N_a$ .

#### 3.2. The file status list

As shown in Fig. 2, the file status table (FSL) is a two-dimensional data structure stored by the CSP that simultaneously sends a copy of the current state of the file state table to the FCC each time it recommends files to the FCC. The FST can be structured in two parts: file information and status information. The file information records the user number and the file number. In a data shared system, the user can be both the data owner and the shared user. The user number is used to identify the files uploaded by different users. The file number is used to identify files that have been uploaded by the same user. For example, if the user number is 8 bits long and the file number is 16 bits long, the system can manage 256 users and the maximum number of files that each user can upload is 65535. In practice, the user number and the file number should be set long enough to satisfy the user's requirements. The other part is the status information, including "LastVST" and "(isAct, isRec)". "LastVST" records the last time the user accessed the file, the initial value is the time when the file was

uploaded; 'isAct, isRec' records whether the file is active or not and whether the file has been recommended to the user respectively. If the file is active, 'isAct' is set to 1, otherwise it is 0. If the file has been recommended to the user, 'isRec' is set to 1, otherwise it is 0. "(0, 0)" indicates that the file is inactive, the file has just been uploaded or has not been accessed within this recommendation cycle  $\Delta T$ ; "(1, 0)" indicates that the file is active but has low access and has not been recommended to the user, the file has a low probability of being accessed within  $\Delta T$  time. "(1, 1)" means that the file has been recommended to the user; "(0, 1)" means that the file status may be wrong and the file needs to be checked or recovered immediately.

## 4. The proposed scheme

The proposed scheme consists of three phases, which are *FileUpload*, *FileRec*, and *FileVerify*. The following are the details:

### 4.1. FileUpload

In this phase, data owner obtains secret key from the PKG, then uses the key to generate the signature for the file, and finally sends the file and the corresponding signature to the CSP.

- (1) *Setup*( $1^k$ )  $\rightarrow$  ( $pp$ ) Input the security parameter  $1^k$ , PKG selects two groups  $G, G_T$  with the same order  $p$ , a generator  $g$  of  $G$ , and a bilinear map  $e = G \times G \rightarrow G_T$ . The PKG randomly selects element  $x, r \in Z_p^*$ , elements  $\mu_1, \mu_2, \dots, \mu_j \in G$ , and the hash function  $H = \{0, 1\}^* \rightarrow G$ . The PKG computes public key  $pk = (h, v) = (g^x, g^r)$ . Output system parameters  $pp = \{g, h, x, \mu, \mu_1, \dots, \mu_j, v\}$ .
- (2) *KeyGen*( $param, ID$ )  $\rightarrow$  ( $sk$ ) Input system the system parameters  $pp$ , PKG generates the system master key  $msk$ . Input  $pp$  and user  $ID$ , PKG generates user private key  $sk$ . the process is as follows. After receiving the user identity  $ID = (ID_1, ID_2, \dots, ID_j)$ , PKG computes the  $sk = (sk', sk'') = \left( \left( \prod_{j=1}^l \mu_j^{ID_j} \right)^x, g^x \right)$ , then sends  $sk$  to the user. The user checks the correctness of the private key  $sk$  and accepts  $sk$  as the user's private key if the following equation holds, otherwise it is not accepted.

$$e(sk', g) = e\left(\prod_{j=1}^l \mu_j^{ID_j}, sk''\right) \quad (1)$$

- (3) *SigGen*( $F, sk, ssk, name$ )  $\rightarrow$  ( $\Phi, \tau$ ) The user generates the signature  $\sigma_i$  for each file block  $m_i \in Z_p^*$  in the file  $F$ . The signature  $\sigma_i$  for file block  $m_i$  is calculated as follows.  $\sigma_i = \left( \prod_{j=1}^l \mu_j^{ID_j} \right)^x \cdot (H(name || i || LastVST) \cdot \mu^{m_i})^r \Phi = \{\sigma_i\}_{i \in [1, n]}$  denotes the set of signatures for the file  $F$ . The user computes  $\tau_0 = name || g^r || g^x$  and computes the file tag  $\tau = \tau_0 || SSig_{ssk}(\tau_0)$  by signing private key  $ssk$ . The user sends  $\{F, \Phi\}$  to the cloud, sends  $\tau$  to the TPA. and finally deletes the local storage information.

### 4.2. FileRec

In this phase, the CSP sets the corresponding tags in the *FST* according to the usage of the files, and calculates the file recommendation for the shared users in the system, and sends

the files with high recommendation to the FCC that provides services for the shared users. When a shared user needs to use a file, first check whether the file has been stored in the FCC, if the file is a recommended file, then directly from the FCC, otherwise from the CSP.

#### (1) *FileAct*( $F, FST$ )

- The file status conversion process is shown in Fig. 4.
- The cloud will store the file uploaded by the user, and the file is inactive, it has not been accessed by any shared user. When the number of accesses to the inactive file reaches the pre-set threshold within the recommended period  $\Delta T$ , the file is activated and the "is-Act" tag in the file status table is set to 1, which makes the file active. The recommendation algorithm calculates the recommendation level of active files for the user, in other words, the algorithm will not calculate the recommendation level of inactive files for the user, which reduces the overhead of computing the recommendation level. Then, the cloud sends the  $N$  most recommended files to the FCC and sets their "isRec" tag to 1. When the user accesses a file, regardless of whether the file is recommended or not, the "lastVst" tag is set as the last access time. When the recommendation degree of a recommended file is no longer the  $N$  files with the highest recommendation degree, the file is turned into an active file, its "isRec" tag is set to 0, and the new recommended file is re-sent to the FCC. When the number of accesses to an active file during the access period is below the preset threshold, the file is converted to an inactive file, "isAct" is set to 0, and the "LastVst" tag is set to the upload time. The file activation process is shown in Algorithm 1.

#### (2) *UserSim*( $F, UserRcd$ ) $\rightarrow$ ( $F_{rcd}$ )

Different recommendation algorithms can be applied depending on the scenario of the scheme and the user access. Since it is difficult to classify files uploaded by users, the proposed scheme uses the Item-based collaborative filtering recommendation algorithms (itemCF) for file recommendation Sarwar (2001). The details of the recommendation algorithm are as follows. The similarity between file  $u$  and file  $v$  is first computed:  $w_{uv} = \frac{\sum_{i \in N(u) \cap N(v)} \log(1 + |N(u)|)}{\sqrt{|N(u)| \cdot |N(v)|}}$ , where  $N(u)$  and  $N(v)$  indicate the number of users who have recently accessed file  $u$  and file  $v$ , respectively. In order to improve the variety, coverage and accuracy of recommendations, the similarity is normalized:  $w'_{uv} = \frac{w_{uv}}{\max_{uv} \cdot w_{uv}}$ , where  $\max_{uv} \cdot w_{uv}$  denotes the maximum value in  $w_{uu}, w_{uv}$  and  $w_{vv}$ . CSP computes the recommendation  $p(u, i) = \sum_{v \in S(u, K) \cap N_u} w_{uv} \cdot r_{vi}$  of the file, where  $S(u, K)$  denotes the  $K$  users whose interests are closest to user  $u$ 's, and  $r_{ui}$  denotes whether user  $u$  has accessed the  $i$  file or not. Since the proposed scheme only needs to compute the recommendation degree of active files,  $r_{ui}$  must be 1. Finally, CSP recommends the  $K$  files with the highest recommendation degree to users. Calculation process of user similarity is shown in Algorithm 2.

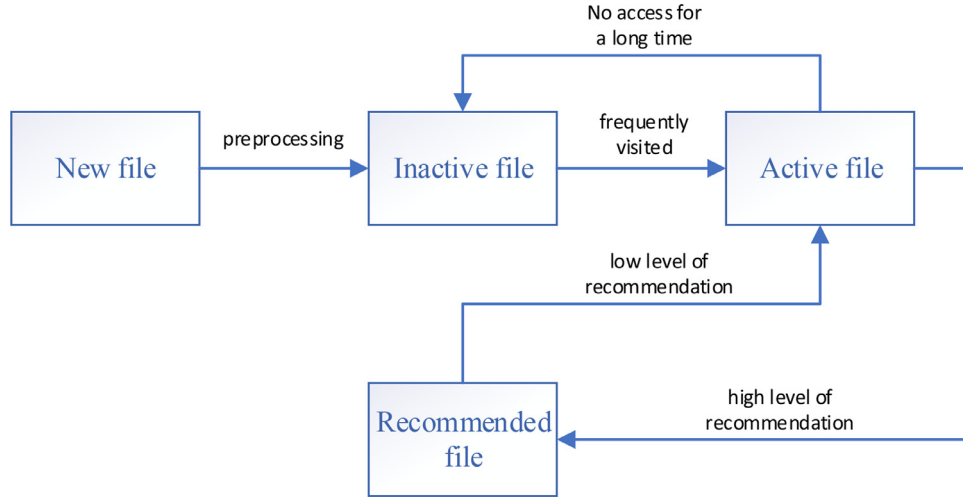


Fig. 4 – The file status list.

#### 4.3. FileVerify

In this phase, users authorizes the TPA to challenge the CSP and then validates the auditing proof returned by the TPA.

- (1)  $ChalGen(FST) \rightarrow (chal)$  After receiving an auditing request from a user, TPA selects files with a valid auditing period and generates a challenge block  $chal$  for these files. If a file is recommended to a user, it means that the file is extremely probable to be accessed by the user during this recommendation period. In other words, if the recommendation algorithm is sufficiently accurate, the recommendation file and the auditing retention period file are equivalent. For example, the steps to generate a challenge  $chal$  for file  $F$  are as follows. The TPA verifies whether the file label  $\tau$  is valid. If  $\tau$  is invalid, the TPA does not continue with the auditing task; otherwise the TPA parses out the filename  $name$  and  $g^r, g^x$ . The TPA checks whether the “isAct” tag and “isRec” tag of file  $F$  are “1” and the time difference between current time and “lastVst” is less than  $\Delta t$ . If not less, then file  $F$  does not need to be audited during the auditing hold period, otherwise generates a challenge  $chal$  for it. The TPA randomly selects a set  $I$  with  $c$  elements,  $I \in [1, n]$ , and a random number  $v_i \in Z_p^*$  for each element  $i$  of  $I$ . The TPA sends the challenge  $chal = \{i, v_i\}_{i \in I}$  to the cloud.

- (2)  $ProofGen(F, \Phi, chal) \rightarrow (P)$

After receiving the challenge sent by the TPA, the cloud generates the auditing proof  $P$  and returns the proof. The details of the steps are as follows. The CSP computes the linear combination of data blocks  $\lambda = \sum_{i \in I} m_i v_i$ . The CSP computes the aggregate signature  $\sigma = \sum_{i \in I} \sigma_i^{v_i}$ . The CSP sends the auditing proof  $P = (\sigma, \lambda)$  to the TPA.

- (3)  $ProofVerify(chal, pp, P)$

The TPA verifies whether the auditing proof is correct by the following process.  $e = (\sigma, g) = e((\prod_{j=1}^l \mu_j^{ID_j})^{\sum_{i=1}^c v_i}, h) \cdot e((H(name||i||LastVST) \cdot \mu^\lambda)^{\sum_{i=1}^c v_i}, v)$  (2)

#### 5. Security analysis

In this section, we demonstrate that the proposed scheme is secure through correctness and soundness.

*Theorem 1 (The correctness)*

- (1) *Private key correctness* : The private key generated by PKG can be passed through the user’s verification.
- (2) *Auditing correctness* : If the cloud stores the user’s data intact, the auditing proof it generates can be validated by the TPA.
- (3) *The correctness of validauditing* : TPA selects files which are in the valid auditing period to audit.

*Proof :*

- (1) The private key  $sk = (sk', sk'')$  generated by PKG can be verified by the equation (1) in the KeyGen algorithm. Based on the properties of bilinear pairing, equation (1) proved as follows.
- (2) The DO’s file  $F$  and the signature corresponding to file  $F$  can be verified by equation (2) in the SigGen algorithm. Based on the properties of bilinear pairing, equation (2) proved as follows.

$$\begin{aligned}
 & e(\sigma, g) \\
 &= e\left(\prod_{i=1}^c \left(\left(\prod_{j=1}^l \mu_j^{ID_j}\right)^{x \cdot v_i} \cdot (H(name||i||LastVST) \cdot \mu^{m_i})^{v_i}\right), g\right) \\
 &= e\left(\prod_{i=1}^c \left(\prod_{j=1}^l \mu_j^{ID_j}\right)^{x \cdot v_i}, g\right) \\
 & \quad \cdot e\left(\prod_{i=1}^c H(name||i||LastVST \cdot \mu^{m_i})^{r \cdot v_i}, g\right)
 \end{aligned}$$

$$\begin{aligned}
&= e\left(\prod_{i=1}^c \left(H(\text{name}||i||\text{LastVST}) \cdot \mu^{m_i}\right)^{v_i}, v\right) \\
&= e\left(\left(\prod_{j=1}^{ID_j} \mu_j^{\sum_{i=1}^c v_i}\right), h\right) \\
&\quad \cdot e\left(\prod_{i=1}^c H(\text{name}||i||\text{LastVST})^{v_i} \cdot \mu^{\lambda}, v\right)
\end{aligned}$$

- (3) If the prediction algorithm is sufficiently accurate, then the recommended files are equivalent to the files that will be used by the user, and then according to the “lastVST” tag in the file status table FST, the files in the valid auditing period can be identified, the audit of these files is a valid auditing.

*Theorem 2 The soundness*

*Auditing soundness:*

- (1) If the CDH problem is difficult in the bilinear map and the signature scheme used to generate file labels is not forgivable. In the proposed scheme, for any cloud or any adversary can generate an auditing proof that can be verified by a TPA if the cloud does not store the user's data intact.
- (2) The replay attack will not work during the validation process. In other words, no dishonest CSP can generate the correct proof to pass the TPA validation using the former information.
- (3) The replace attack also will not be effective during the validation process. In other words, if the CSP does not store the challenge block information correctly, it cannot pass the TPA validation using the proof generated from the unchallenged block information.

*Proof:*

- (1) Suppose that the proof generated by the cloud still passes validation in case of data corruption, the complete data block can be obtained by repeated interaction between the proposed scheme and the constructed a knowledge extractor. This problem was proved by Boneh et al. (2004), followed by a detailed description in Shen et al. (2019) and Li et al. (2019), which will not be repeated here.
- (2) Replay attacks usually occur during the verification phase and to be defined as follows. The data owner or user authorizes the TPA to challenge the CSP to check whether the cloud files are intact. After getting user authorization TPA first selects a number of files which are in the valid auditing period, and then randomly selects  $s$  blocks of files to generate challenge blocks respectively. Take file  $F$  as an example, the TPA generates the challenge block  $chal = \{i, r_i\}_{i \in [1, s]}$  for it and sends it to the CSP. However, the CSP attempts a replay attack. the CSP replaces some of the information in this auditing using information from the previous auditing. Suppose the CSP replaces the information of the  $j$ -th file block with the information of the previous block, the information of the  $m_j$  block is replaced by the information of

$m_{j-}$ . In other words, the CSP generates the illegal proof  $P^- = (\sigma^- = (\sigma^-, \lambda^-))$  and sends it to the TPA. where the aggregate signature  $\sigma^-$  is

$\sigma^- = \prod_{i \in [1, k-1] \cup [k+1, s]} \sigma_i^{v_i} \cdot \sigma_{k-}^{v_{k-}}$  The linear combination of data blocks  $\lambda^-$  is  $\lambda^- = \sum_{i \in [1, k-1] \cup [k+1, s]} m_i \cdot v_i + m_{k-} \cdot v_{k-}$ . After receiving the proof  $P^-$ , the TPA verifies it. If the proof  $P^-$  passes the verification, the CSP has successfully performed a replay attack; otherwise, the replay attack fails. To be more convincing, substitute the proof  $P^-$  into equation (2) to show that replay attacks cannot succeed. Substitute the aggregate signature  $\sigma^-$  into the left side of equation (2).

$$\begin{aligned}
&e(\sigma^-, g) \\
&= e\left(\prod_{i \in [1, k-1] \cup [k+1, s]} \left(\prod_{j=1}^k \mu_j^{ID_j}\right)^x \cdot (H(\text{name}||i||\text{LastVST}) \cdot \mu^{m_i})^r \cdot \left(\prod_{j=1}^l \mu_j^{ID_j}\right)^x \cdot (H(\text{name}||k||\text{LastVST} \cdot \mu^{m_{k-}})^r)^{v_i}, g\right) \\
&= e\left(\prod_{i=1}^s \left(\prod_{j=1}^l \mu_j^{ID_j}\right)^{x \cdot v_i}, g\right) \\
&\quad \cdot e\left(\prod_{i \in [1, k-1] \cup [k+1, s]} (h(\text{name}||i||\text{LastVST}) \cdot \mu^{m_i})^{r \cdot v_i} \cdot H(\text{name}||k||\text{LastVST})^{r \cdot v_{k-}}, g\right) \\
&= e\left(\prod_{i=1}^s \left(\prod_{j=1}^l \mu_j^{ID_j}\right)^{v_i}, h\right) \\
&\quad \cdot e\left(\prod_{i \in [1, k-1] \cup [k+1, s]} (H(\text{name}||i||\text{LastVST}) \cdot \mu^{m_i})^{v_i} \cdot (H(\text{name}||k||\text{LastVST}^-) \cdot \mu^{m_{k-}})^{v_{k-}}, v\right)
\end{aligned}$$

Substitute the linear combination  $\lambda^-$  into the right side of equation (2).

$$\begin{aligned}
&e\left(\left(\prod_{j=1}^l \mu_j^{ID_j}\right)^{\sum_{i=1}^c v_i}, h\right) \\
&\quad \cdot e\left(\prod_{i=1}^c H(\text{name}||i||\text{LastVST})^{v_i} \cdot \mu^{\lambda^-}, v\right) \\
&= e\left(\left(\prod_{j=1}^l \mu_j^{ID_j}\right)^{\sum_{i=1}^c v_i}, h\right) \\
&\quad \cdot e\left(\prod_{i \in [1, k-1] \cup [k+1, s]} (H(\text{name}||i||\text{LastVST}) \cdot \mu^{m_i})^{v_i} \cdot (H(\text{name}||k||\text{LastVST}^-) \cdot \mu^{m_{k-}})^{v_{k-}}, v\right)
\end{aligned}$$

Comparing the two equations above shows that a replay attack can pass verification only if  $\lambda^- = \lambda$ , that is,  $k = k^-$  and  $\text{LastVST} = \text{LastVST}^-$ , which is less than  $1/p$ . In other



words, a CSP cannot perform a replay attack to pass a TPA with a non-negligible probability.

- (3) We define the replacing attack as follows. Similar to the replay attack, the DO or user authorizes the TPA to challenge the CSP, and then the TPA selects the files which are in the valid auditing period to be audited, and then generates a challenge block for each file. At this time, CSP is conducting a replacing attack. Suppose that the CSP replaces the  $r$ -th file block  $m_r$  with file block  $m_{r-}$  and generates an audit certificate  $P' = (\sigma', \lambda')$  using the information from the replaced file. Where the aggregate signature  $\sigma'$  is.

$$\sigma' = \prod_{1 \leq i \leq k-1 \cup [k-1, s]} \sigma_i^{v_i} \cdot \sigma_{k-}^{v_{k-}}$$

and the linear combination is

$$\lambda' = \sum_{i \in [1, k-1] \cup [k-1, s]} m_i \cdot v_i + m_{k-} \cdot v_{k-}$$

The verification process is similar to a replay attack and is not repeated here. the probability that the proof  $P'$  forged by the CSP passes verification is  $1/(n \cdot p)$ . In other words, the proof forged by the CSP cannot pass verification with a non-negligible probability.

**Theorem 3(The Detectability):** Assume that file  $F$  is divided into  $n$  blocks and stored in the cloud. The  $k$  blocks are modified or deleted by the CSP, and the number of challenge blocks is  $c$ . At this time, the detectability of the PPABC protocol is  $\left(\frac{k}{n}, 1 - \left(\frac{n-k}{n}\right)^c\right)$ .  $X$  is the intersection of the damaged block and the challenge block. We have the following proof:

$$\begin{aligned} Px &= P\{X \geq 1\} \\ &= 1 - P\{X = 0\} \\ &= 1 - \frac{n-k}{n} \times \frac{n-1-k}{n-1} \times \dots \times \frac{n-c+1-k}{n-c+1} \end{aligned}$$

## 6. Performance evaluation

In this section, the performance of the proposed scheme will be presented. In section 7.1, the results of user behaviour prediction experiments and a valid audit experiment are presented. In section 7.2, a functional comparison of the proposed scheme with related schemes is presented. In section 7.3, the comparison of the computational overheads of the proposed scheme with those of the related scheme is presented, and in section 7.4, the communication overheads and the computational complexity of the proposed scheme at different phases are presented.

### 6.1. User behavior prediction experiments

The experiment used the dataset UserBehavior.csv provided by Aliyun Han et al. (a, b). The dataset includes records of about 1 million users accessing 4 million items in a week. Since in practice a typical data sharing system serves roughly

between 10 and 1000 users, we divided the dataset into groups of 100 users to obtain a total of 10,000 sets of data. Of these, 75% were used as train sets and 25% as test sets for experimentation. The precision is 0.647, the recall rate is 0.754, and the coverage rate is 0.468. This means that the Item-based collaborative filtering recommendation algorithm is proven to be effective in data sharing scheme. The experimental results are shown in Table 3.

### 6.2. Functionality comparison

This section gives a comparison of the functions of the proposed scheme and related schemes Huang (2020) Shen et al. (2019) Li et al. (2019). As shown in Table 2, the proposed scheme is the only one that supports the following properties: public auditing, certificate management simplification, data sharing, and data recommendation. The proposed scheme is the only one among the above schemes that supports data recommendation.

### 6.3. Performance analysis and comparison

$Hash_G$ ,  $EXP_G$  and  $Mul_G$  denote a hash operation, an exponential operation, and a multiplication operation on group  $G$ , respectively. Similarly,  $Mul_{Z_p}$  and  $Add_{Z_p}$  denote a multiplication operation, and an addition operation in  $Z_p^*$ , respectively.  $P$  denotes a pairing operation.  $l$  denotes the length of the user's identity information.  $l$  denotes the length of the user's identity information.  $|n|$  denotes the length of the set  $[1, n]$ .  $|p|$  is the length of the element in  $Z_p^*$  and  $|q|$  represents the length of the element in  $G$ .  $c_1$  represents the challenge block required by the proposed scheme,  $c_2$  represents the challenge block required by the existing scheme.

1) **Computation overhead comparison** Table 4 shows the cost comparison between the proposed scheme and the related schemes. Chapter 4 presents the details of the operation of the proposed scheme's phases. The computational cost for the signature generation is  $n - (H + 2\varepsilon + 2Mul)$ , the computational cost for the auditing proof generation is  $c(Add + Mul + \varepsilon)$  and the computational cost for the proof verification is  $c - (Add + \varepsilon) + Mul(2c + 1) + 3P$ . As shown in Table 2, the cost of generating the signature for a given file and the cost of generating the proof is  $c(Add + Mul + \varepsilon)$ . The computation of the cost of validation was not significantly different between the proposed and the existing schemes. However, since the proposed scheme only audits files which are in the valid auditing period, obviously  $c_1$  is much smaller than  $c_2$ , the overall auditing cost is less than the other existing schemes. This is described in detail in section 7.4 of this chapter.

2) **Communication overhead comparison:** Based on the detailed description of the scheme in Chapter 4, it is shown that the major communication overhead of the proposed scheme is concentrated in the auditing phase, therefore, only communication In the integrity auditing phase, the TPA first selects the files that are in the valid auditing period based on the tags in the FST and generates a challenge block  $chal = \{i, v_i\}_{i \in I}$  for them, respectively. The size of the challenge block is  $c \cdot (|n| + |p|)$  bits. After receiving the challenge from the TPA, the cloud generates the auditing proof  $P = \{\lambda, \sigma\}$  and sends it back to the TPA. the size of the auditing proof  $P$  is  $|p| + |q|$  bits.

**Table 2 – FUNCTIONALITY COMPARISON WITH EXISTING RELATED SCHEMES.**

|                                                | Public Verifiability | Certificate management simplification | data sharing | File recommended | Lightweight supported |
|------------------------------------------------|----------------------|---------------------------------------|--------------|------------------|-----------------------|
| Huang et al. <a href="#">Huang (2020)</a>      | Yes                  | Yes                                   | No           | No               | Yes                   |
| Shen et al. <a href="#">Shen et al. (2019)</a> | Yes                  | Yes                                   | Yes          | No               | No                    |
| Li et al. <a href="#">Li et al. (2019)</a>     | Yes                  | Yes                                   | No           | No               | No                    |
| Ours                                           | Yes                  | Yes                                   | Yes          | Yes              | Yes                   |

**Table 3 – THE RESULTS OF USER BEHAVIOR PREDICTION.**

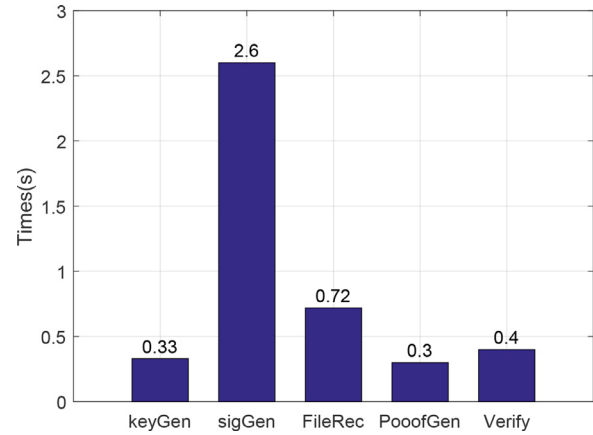
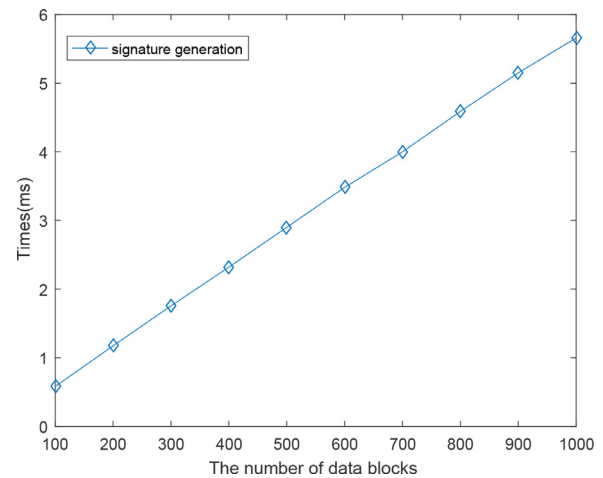
|           |       |
|-----------|-------|
| Precision | 0.647 |
| Recall    | 0.764 |
| coverage  | 0.468 |

Therefore, the total communication overhead of the auditing is  $c \cdot (|n| + |p|) + |p| + |q|$ .

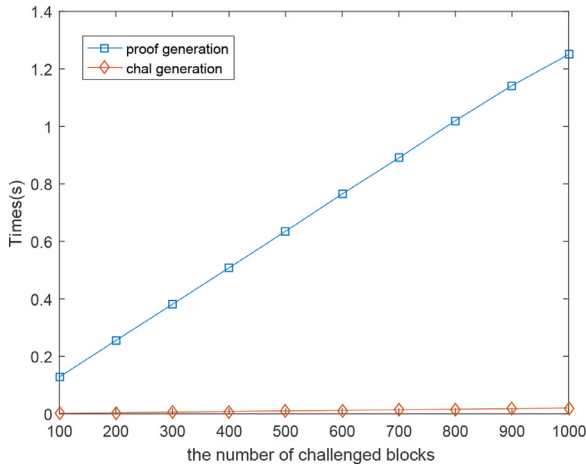
#### 6.4. Experimental results

In this section, we will present and evaluate the experimental results of the proposed scheme. The experiments are run in a Linux virtual machine environment. A single-core 2.0 GHz CPU and 2 GB of RAM were assigned for the VM. All algorithms were implemented using C language along with PBC [Lynn \(2015\)](#) and GMP [2017](#) programming. In the experiments, this paper sets the domain size to 512bit and the element size  $|p|$  in the field is 160bit. the file size is 20MB and contains 10,000 data blocks. The length of the user identity information is 160 bit.

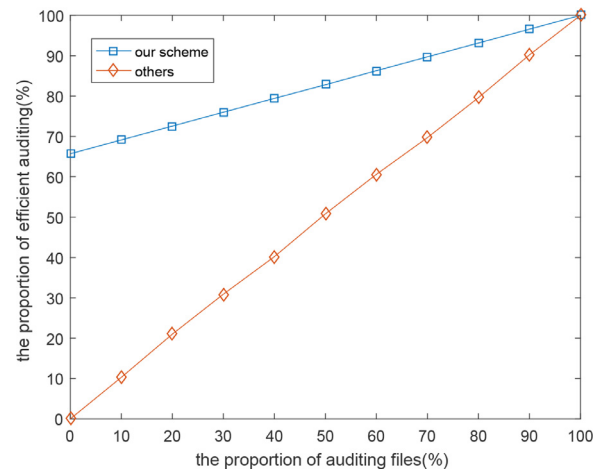
1) *performance of Different Process*: To present the experimental results more clearly, let the data block be 100. The experimental results are shown in [Fig. 5](#), the computational overhead of generating the private key and verifying the private key is 0.33s, the overhead of signature generation is 2.6s, the computational overhead of proof generation is 0.3s, and the computational overhead of proof verification is 0.4s. from this, it can be seen that the overhead accounts for the largest percentage of the signature generation phase, and the overheads of all other phases are smaller. [Fig. 6](#) shows the change in overhead for signature generation. When the data block increases from 0 to 1000, the computational overhead for signature generation increases from 5.82ms to 56.59ms. [Fig. 7](#) shows the overhead for challenge generation and proof generation when the challenge block increases from 0 to 500. The chal-

**Fig. 5 – Performance of different processes.****Fig. 6 – The computation overhead in process of signature generation.****Table 4 – THE COMPUTATION OVERHEAD OF OUR SCHEME AND RELATED SCHEME IN DIFFERENT PHASES.**

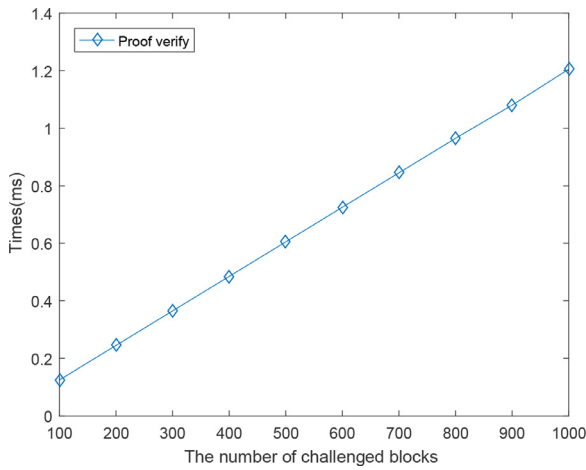
|                 | SigGen                           | ProofGen                                                               | Verify                                                                         |
|-----------------|----------------------------------|------------------------------------------------------------------------|--------------------------------------------------------------------------------|
| Huang's Scheme  | $(1 + 2n)Mul + 3n\epsilon + np$  | $(c + w)\epsilon + (2c_2 - 1)Mul$                                      | $(c_2 + 1)\epsilon + (c_2 - 1)Mul + 2P$                                        |
| Li's Scheme     | $n \cdot (H + Mul + 2\epsilon)$  | $c \cdot Mul + c_2 \cdot \epsilon$                                     | $c_2 \cdot (\epsilon + H) + w \cdot (2P + \epsilon + Mul)$                     |
| Shen's Scheme   | $n \cdot (H + 2Mul + 2\epsilon)$ | $(2c_2 - 1) \cdot Mul + c_2 \cdot \epsilon +$<br>$(c_2 + 1) \cdot Add$ | $4P + (2 + c_2 + l)Mul +$<br>$2(c_2 - 1)Add + (l + c_2)\epsilon + c_2 \cdot H$ |
| purposed Scheme | $n \cdot (H + 2\epsilon + 2Mul)$ | $c_1 \cdot Add + (2c_1 + 1)Mul + c_1 \cdot$<br>$\epsilon + 3P$         | $2P + (2c_1 - 1)Mul + 2\epsilon +$<br>$2cH + (2c_1 - 1)Add$                    |



**Fig. 7 – The computation overhead in process of challenge generation and proof generation.**



**Fig. 9 – The comparison of valid auditing.**



**Fig. 8 – The computation overhead in process of proof verification.**

challenge generation overhead increases from 2ms to 20ms, and the auditing proof overhead increases from 0.128s to 1.28s. Fig. 8 shows the proof validation overhead. When the challenge block is increased from 0 to 500, the proof verification overhead is increased from 0.125s to 1.205s.

2) *performance of valid auditing*: This section shows in detail the auditing overhead of the proposed scheme compared with the existing scheme following the introduction of the concept of “valid auditing”. In the proposed scheme, accuracy is positively correlated with the accuracy of valid audits, as well as the predicted accuracy and the proportion of files which are in the valid auditing period. On the other hand, in the existing scheme, the auditing tends to be random and the valid auditing rate depends exclusively on the proportion of documents in the valid auditing period. Fig. 9 shows the change in the valid auditing rate as the proportion of files within the valid audit period increases from 10 to 100 percent. Fig. 9 shows that the actual valid auditing rates of the proposed schemes are better than the existing schemes. The difference is more obvious when the proportion of files within the valid auditing

---

**algorithm 1** FileAct.

---

**Require:** File  $F$ , User  $user$

```

1: function FILEACT( $F$ ,  $user$ )
2:   if  $count > Count_{\Delta T}$  then
3:      $isAct$  to 1
4:      $UserSim()$ 
5:   end if
6: end function
7:
8: function FILEVISIT( $F$ ,  $User$ )
9:    $set F$ 's lastVST to 1
10:  if recommended level is not Top K then
11:     $set State$  to ActFile
12:     $set isRec$  to 0
13:     $send recommended files$  to FCC
14:  end if
15: end function
16:
17: function FILEVISIT( $F$ )
18:  if  $count < Count_{\Delta T}$  then
19:     $set isAct$  to 0
20:     $set LastVst$  to Upload Time
21:  end if
22: end function

```

---



---

**algorithm 2** UserSim.

---

**Require:** User  $user$ , File  $F$

```

1: function USERSIM( $user$ ,  $F$ )
2:   $compute W_{uv} W'_{uv}$  and  $p(u, i)$ 
3:   $send K files$  with the highest recommendation
4:   $degree$  to users
5: end function

```

---

period is lower. In practice, the proportion of files in the valid auditing period was generally not higher than 50 per cent, so that the valid auditing rate of the proposed scheme was on average 45 per cent higher than that of the existing scheme.

## 7. Conclusion

In this paper, we propose an user behavior prediction-based data integrity auditing scheme for secure cloud storage. The recommended files are stored in the fog nodes for users to access, which reduces the access delay and improves the user experience. In addition, this paper introduces the concept of “the valid auditing”, which selects the files in the valid auditing period according to the tags in the file status table to reduce the number of files to be audited. The security analysis shows that the scheme is secure and reliable. The experimental results show that the user behavior prediction algorithm is accurate and effective if the parameters are set correctly. It also reduces the auditing overhead because the valid auditing only needs to be performed on the files which are in the valid auditing period.

## Declaration of Competing Interest

The Author(s) declare(s) that there is no conflict of interest.

## CRediT authorship contribution statement

**Junfeng Tian:** Supervision, Project administration, Funding acquisition, Writing - review & editing. **Haoning Wang:** Conceptualization, Investigation, Writing - original draft. **Meng Wang:** Data curation.

## REFERENCES

- Ateniese G. Provable data possession at untrusted stores. in Proc 14th ACM Conf Comput Commun Secur 2007:598–609.
- Boneh, Dan, Franklin M. In: *Advances in Cryptology CRYPTO 2001. Identity-based Encryption from the Weil Pairing*. Springer Berlin Heidelberg; 2001.
- Boneh D, Lynn B, Shacham H. Short signatures from the weil pairing. *J Cryptology* 17 2004:297–319. doi:10.1007/s00145-004-0314-9.
- Bonomi F, Milito R, Zhu J. Fog computing and its role in the internet of things. *Proceedings of the 1st Edition of the MCC workshop on mobile cloud computing, Helsinki, 13–17 August 2012*;13–16. New York: ACM
- Dewan H, Hansdah RC. A survey of cloud storage facilities. *Proc 7th IEEE World Congress on Services* 2011:224–33.
- Erway C, Kupcu A, Papamanthou C, Tamassia R. Dynamic provable data possession. in *Proc 16th ACM Conf Comput Commun Secur* 2009:213–22.
- GNU T. Multiple precision arithmetic library. (GMP) 2017. Online <http://gmplib.org>
- Han Z., Daqing C., Ziru X., et al. Joint optimization of tree-based index and deep model for recommender systems. a. ArXiv:1902.07565.
- Han Z., Xiang L., Pengye Z.. Learning Tree-based Deep Model for Recommender Systems. In: *In Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery Data Mining*.
- Hogan M, Liu F, Sokol A, Tong J. NIST Cloud computing standards roadmap. NIST Cloud Computing Standards Roadmap Working Group SP 500-291-v10, NIST 2011.
- Huang K. EVA: Efficient versatile auditing scheme for iot-based datamarket in jointcloud. in *IEEE Internet of Things Journal* 2020;7(2):882–92. doi:10.1109/JIOT.2019.2945921.
- Igeltjorn A, Habib L. In: In: Antona M., Stephanidis C. (eds) *Universal Access in Human-Computer Interaction. Applications and Practice. HCII 2020. Lecture Notes in Computer Science*, vol 12189. Homebased Telework as a Tool for Inclusion? a Literature Review of Telework, Disabilities and Work-life Balance. Springer, Cham; 2020.
- Juels A, Kaliski JBS. Pors: proofs of retrievability for large files. *Proc 14th ACM Conf Comput Commun Secur* 2007:584–97.
- Li Y, Yu Y, Min G. Fuzzy identity-based data integrity auditing for reliable cloud storage systems. *IEEE T Depend Secure Comput* 2019;16:72–83.
- Lynn B.. The pairing-based cryptographic library. 2015. [Online]. Available: <https://crypto.stanford.edu/pbc>.
- Ma Y, Liang W, Wu J, Xu Z. Throughput maximization of NFV-enabled multicasting in mobile edge cloud networks. in *IEEE Transactions on Parallel and Distributed Systems* 2020;31(2):393–407. doi:10.1109/TPDS.2019.2937524.
- Ren K, Wang C, Wang Q. Security challenges for the public cloud. *IEEE Internet Comput* 2012;16(1):69–73.
- Sarwar B. Item-based collaborative filtering recommendation algorithms[c. *Proceedings of the 10th International Conference on World Wide Web New York: WWW10* 2001:285–95.
- Shen J, Shen J, Chen X, Huang X, Susilo W. An efficient public auditing protocol with novel dynamic structure for cloud data. in *IEEE Transactions on Information Forensics and Security* 2017;12(10):2402–15. doi:10.1109/TIFS.2017.2705620.
- Shen W, Qin J, Hao R. Enabling identity-based integrity auditing and data sharing with sensitive information hiding. *IEEE Trans Inf Forensics Security* 2019;14(2):1556–6013.
- Tian H. Dynamic-hash-table Based Public Auditing for Secure Cloud Storage, volume volume 10; 2017. p. 701–14. doi:10.1109/TSC.2015.2512589.
- TIAN H, NAN F. Privacy-preserving public auditing for secure data storage in fog-to- cloud. *Journal of Network and Computer Applications* 2019;127:59–69.
- Wang C, Chow SSM, Wang Q, Ren K, Lou W. Privacy - preserving public auditing for secure cloud storage. *IEEE Trans Comput* 2013;62(2):362–75.
- Wang Q, Wang C, Ren K, Lou W, Li J. Enabling public auditability and data dynamics for storage security in cloud comput- ing. *IEEE Trans Parallel Distrib Syst* 2011a;22(5):84–859.
- Wang Q, Wang C, Ren K, Lou W, Li J. Enabling public auditability and data dynamics for storage security in cloud computing. *IEEE Trans Parallel Distrib Syst* 2011b;22(5):847–59.
- Worku SG, Xu C, Zhao J, He X. Secure and efficient privacy-preserving public auditing scheme for cloud storage. *Comput Electr Eng* 2014;40(n5):1703–13.
- Yu J, Wang H. Strong key-exposure resilient auditing for secure cloud storage. *IEEE Trans Inf Forensics Security* 2017;12(8):1931–40.



**Junfeng Tian** Professor, Deputy Secretary General, Open System Specialized Committee of China Computer Federation; Young and Middle-aged Specialist with Outstanding Contribution in Hebei Province; Young and Middle-aged Teacher with Outstanding Contribution in Hebei Province; He graduated from Hebei University in 1986, received a master's degree in computer science from Xi'an University of Electronic Science and Technology in 1990–1991 and a master's degree in 1995. In 2004, he graduated from the University of Science and Technology of

China with a major in computer science and technology. He has been engaged in teaching and research in the fields of distributed computing and network technology for many years.



**Haoning Wang** He was born in 1996. He is a master graduate student. His main research interests include security of information and trusted compute, distributed computation.



**Meng Wang** Associate Professor. He received the Ph.D. degree in computer science from Xidian University, Xi'an, China, in 2019. His research interests include model checking, temporal logics, and formal verification of software systems. He has been engaged in teaching and research in the fields of cyberspace security.